structure requires any thread to acquire a lock on it (B) before being able to write a value (C). If another thread tries to acquire the lock while it is still in use by thread `N`, it will be left waiting till the lock is released (D).
After that (not shown in the figure), the waiting thread can acquire the lock, or possibly compete with other threads trying to acquire it.

Modern programming languages provide plenty of mechanisms to ensure thread-safety, from locks to semaphores to latches.

An extensive covering of this topic would require a book on its own, so here we will have, unfortunately, to make do with an example: we'll show how to solve this problem in Java, providing the full code for thread-safe LRU/LFU in our repo.

7.7.1 Solving Concurrency (in Java)

If we are making another exception in this chapter, showing some code in a specific programming language rather than pseudo-code, it's because we feel that these concepts can only by tackled effectively within the concrete context of a real language.

As many programming languages support the same concepts, it should be easy to port this logic to your favorite environment.